

Agents 重构 金融 FICC 策略全生命周期管理平台

演讲嘉宾：陈治平

华泰证券固定收益部 策略管理团队负责人

先做个自我介绍

金融科技一线二十年，亲手交付三套电子交易平台。以下是三个里程碑项目：

1

2019 平安证券

「领航 FLEET」固收外汇电子做市系统

深圳市金融创新奖一等奖

2

2022 华泰证券

「大象 FICC 多资产做市与风控平台」

人民银行金融科技发展奖一等奖

3

2026 AURORA 平台

「AI + AURORA 策略管理平台」

AI Agents 嵌入多资产策略全生命周期管理

4000亿

日均成交

50万

日均订单

98%

策略下单占比

- ✓ 2024 微调GLM落地智能债券询价机器人，AI 做市业务量三年超 20 倍增长（2024 年 396 亿 → 2025 年 2,000 亿 → 2026 上半年 4,600 亿）
- ✓ 2025 基于玻璃箱v1架构构建策略全生命周期管理，把Agent引入金融研究、因子挖掘、策略编写、交易辅助、执行复盘场景
- ▨ 2026 重构策略全生命周期管理平台（进行中，本次分享内容）

目录

CONTENT

01 重构背景

02 双区架构

03 三板斧：轻编排、自
进化、生成式 UI

04 工程心得分享

01

重构背景

为什么要重构

第一代解决"能不能用", 第二代回答"能不能越用越强"

📅 今天的主线——第二代的三个新问题

✅ 第一代 已在生产

玻璃箱 + 无侵入上线。
核心命题：**能不能用**。
已验证可行，正在生产环境稳定运行。

🔧 第二代 正在推广

初步跑通，正在推广。"
让 Agent 跑起来"不再是
难点，难点是**越用越强**。

01

编排为什么要变薄

从刚性脚手架到轻量 harness，模型能力提升后编排层的演化逻辑

02

Agent 怎么自进化

受监管场景下，比 Karpathy 再收紧一格的金融Agent自进化设计

03

AI 产出如何固化成可信的分析应用

金融原生 GenUI + 确定性管线，幻觉进不了数据层

02

双区架构

核心不是模型选型，是边界划分

常见的错误顺序

随着AI技术的突飞猛进，金融行业对AI的信心初步建立

很多金融机构先纠结选哪个模型——GPT-4o 还是 Claude? 开源还是闭源?

这个顺序反了。

边界先于模型。模型是可以替换的，边界划错不可逆。

为什么边界比模型更重要

- **98% 订单由策略下单**，一个错误动作影响面极大。在这个量级上，没有"小错误"。
- 边界划错，模型再强只是**放大系统性风险**。更强的模型 = 更大的爆炸半径。
- **模型可以替换，边界划错不可逆**。这是整个架构设计的第一原则。

双区架构：严控“爆炸半径”



研发分析区

- 自由探索试错，错了进不了交易，代价可控。
- AI 的不确定性被限制在这一侧。

交易核心链路

- 不容 AI 不确定性渗入，一动就是真金白银。
- 执行器只认签名许可，不认自然语言。

唯一的桥

- 回测 + 上线流程。自由区无生产写权限，只能提交结构化 ActionProposal。
- 核心侧独立校验后，签发带签名的 ExecutionPermit。

边界定义：不是"谁是 AI"，而是动作风险

架构可以扩展到不同的业务，按副作用、可逆性、影响范围、可验证性分四级。锚的是动作风险，不是"谁点的按钮"。

L0 只读生成 自由，无需审批。生成报告、查询数据、归因分析。副作用为零。	L1 可逆低风险写 校验通过自动执行。参数调整、草稿保存等可撤销操作。影响范围可控。
L2 受控业务写 需审批。策略上线、额度变更等业务影响的操作。需人工确认。	L3 高风险不可逆 Agent 只能提案，永远不直接执行。大额交易、系统配置变更等。人在环，不可绕过。

这套分级逻辑不只适用于金融。医疗、法律、监管型行业同样适用——核心是按动作的不可逆程度划定自治边界。

等级	动作示例	Agent 权限	控制与回退
L0 生成/只读	检索、摘要、报告草稿、可视化编排	可自主	数据最小化、只读凭证;结果不进正式记录
L1 可逆低风险写	草稿工单、标签、内部通知、缓存	策略通过后自动	幂等键、范围/数量上限、可撤销、事后抽样审计
L2 受控业务写	CRM 字段、报告发布、非资金权益变更	需结构化审批	审批绑精确参数、短令令牌、四眼原则、补偿事务
L3 高风险/不可逆	交易、付款、授信拒绝、客户承诺、生产配置、提权	只能提案	独立固定 UI、多人授权、确定性规则引擎、预交易风控、fail-closed、不可篡改账本

唯一通道：ActionProposal → ExecutionPermit



AGENTIC AI
超级智能体系统架构峰会

✓ 生产实证

🔧 审计链在建

铁律：执行器只认签名许可，不认自然语言。上游有注入、有幻觉，也越不过这道签名的闸。

1 提交 ActionProposal

自由区无生产写凭证，只能提交结构化 ActionProposal，不能直接操作生产系统

2 独立校验

核心侧独立校验：身份、策略、参数、额度、审批——五项缺一不可

3 签发 ExecutionPermit

校验通过，签发带签名的 ExecutionPermit，包含完整的审计信息

4 执行器验签

执行器只认许可，自然语言指令一律不接。签名是唯一凭证

📌 这道签名闸的工程刚性是整个系统安全性的最后一道防线。任何绕过它的“便捷方式”都是系统性风险的入口。

玻璃箱 v2：管边界，不管步骤

□ 呈现的是可审计的理由链，不是模型原始思维流。

01

意图

用户或系统的原始目标

02

引用

调用的数据源和工具

03

工具结果

确定性管线的输出

04

策略判定

Agent 的推理与决策

05

签名许可

ExecutionPermit, 可审计

玻璃箱呈现的理由链

玻璃箱 v1 管每一步

刚性脚手架框住弱模型。每一个步骤都有明确的编排约束，适合模型能力不足时的保守策略。

已在生产验证，稳定运行。

玻璃箱 v2 管边界，放自主

够强的地方自主编排，够弱的地方保留刚性（参考 arXiv:2603.28990）。

注意：这不是"结构越少越好"的通用结论，只在模型能力达标处放自主。

呈现的是可审计的理由链，不是模型原始思维流。

03

三板斧

一：编排做薄，落到开源 agent harness

实证依据

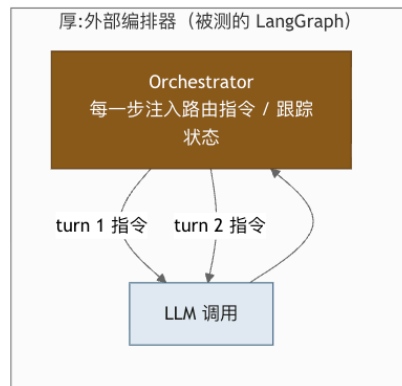
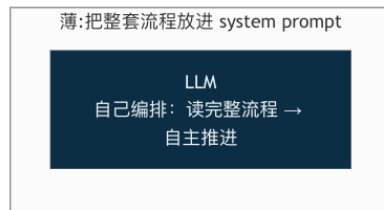
arXiv:2604.27891 实证：程序化任务 + 前沿模型 + 实测 LangGraph 条件下，
"外部编排已非必要"。模型能力提升后，厚重的编排层反而成为约束。

做法

新建编排层落到 **Hermes-agent** (MIT 开源，同类 OpenClaw)。轻量、
可审计、可替换。

Hermes 同时是自进化载体

- **skills 累积** curator 管理生命周期，技能库随使用不断丰富
- **/goal 驱动** 标准目标接口，统一任务描述格式
- **checkpoints + rollback** 提个改动，不行就回退。工程安全兜底



"薄" = 把流程写进 system prompt、让模型自编排;"厚" = 在模型之上架一层每回合注入路由的外部编排器 (论文实测对象是 LangGraph)。

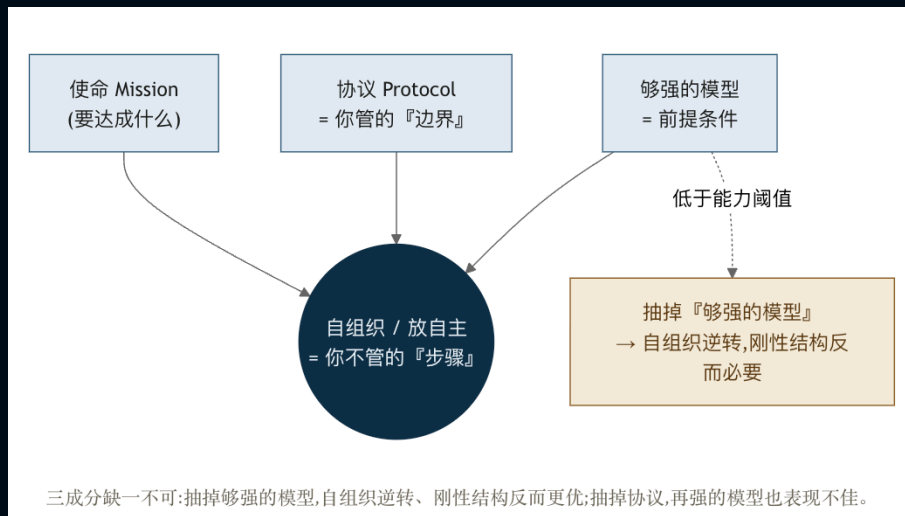
一个反例，恰好证明纪律

✓ 生产实证

智能债券询价机器人至今跑在 LangChain 上

没有迁移到 Hermes，也没有计划迁移。

换框架要有业务理由，不能追时髦。这是一个刻意保留的反例，用来提醒团队：技术决策必须由业务需求驱动，而不是由新框架的热度驱动。



开源红利的正确用法

✓ 红利是"试得起"

新能力低成本试起来：新 provider、记忆后端、MCP，拿来就能试。失败成本低。

⊘ 不是"拿来就上"

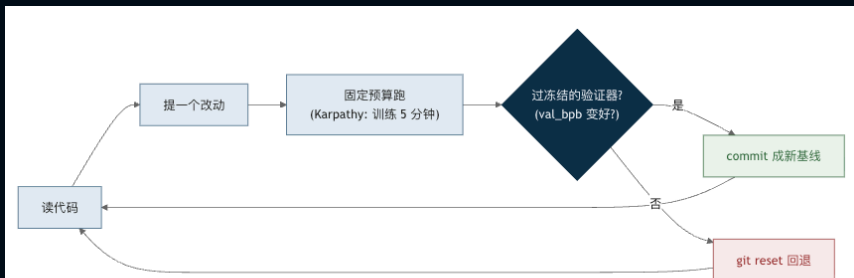
进核心链路必须过闸：

Scorecard、SLSA、BOM、签名，一个都不能少。

二：金融的自进化需要再收紧

借鉴：Karpathy Autoresearch 循环

arXiv:2603.23420: 提改动 → 固定预算跑一轮 → 过验证器就 commit, 没过就 git reset。



发动机是那个冻结、不可作弊的验证器。Karpathy 的话:"有了客观指标,跑实验的人就不该是你——把自己从循环里拿掉。"反过来:没有验证器,你只是花钱雇了个整晚给自己点头的 agent。

金融场景的收紧

搜索空间只有三样

prompt、skill、路由——仅此三项, 不多一样

模型权重 + 验证口径冻结

Karpathy 改训练代码, 我们方向相反。权重不动, 验证标准不动



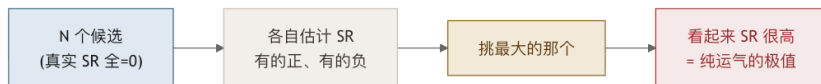
同一套 loop, 搜索空间对调。Karpathy 冻考卷、放模型: 我们冻两样(模型权重 + 验证口径)、只放 prompt/skill/路由——更窄、更保守。

📌 定位: 离线的、受约束的候选生成器, 不是"系统自我进化"。任何改动都算系统变更, 照走验证和回退流程。

小心金融指标进化变成“过拟合表演”

Sharpe 是估计,挑"最好的"必然虚高

sharpe 比率 $SR = \text{均值}/\text{波动}$ 是从有限样本算出来的估计量,自带估计误差。哪怕一个策略的真实 $SR = 0$,在一段历史上也会算出一个非零的 SR ——有时正、有时负。



"挑最好的" = 取 N 个随机估计的极大值。极大值随 N 增长——所以自进化跑很多候选时,这个虚高被放到最大。

必须校验 Deflated Sharpe Ratio

$$DSR = P(\text{真实 } SR > E[\max SR \text{ under } N \text{ trials}])$$

它同时吃掉了两个放大器:分母里的偏度/峰度(非正态)、基准里的 N 与 σ_{SR} (多重检验)。DSR 低(如 < 0.95) → 这个"漂亮回测"很可能只是多重检验的运气极值,不是技能。

多重检验:要打败的基准不是 0

冻结验证集会慢慢退化成训练集。验证口径本身需要定期审视,但不能频繁变动。

非正态让 Sharpe 的误差更大

SR 估计量的标准误差不只看样本长度,还看收益分布的偏度 γ_3 与峰度 γ_4 , 正态假设下"看着显著"的策略,换成真实分布可能根本不显著。

四个核心杠杆

$N \uparrow$	试验次数越多 → 基准越高 → 越难为真
$n \downarrow$	样本越短 → 估计误差越大 → 越难为真
负偏 γ_3	→ 抬高误差 → 越难为真
肥尾 γ_4	→ 抬高误差 → 越难为真

验证器，才是最要害的一环

工程底线：验证器不能被作弊

跑循环不难，难的是验证器的设计。一个可以被作弊的验证器，等于没有验证器。

- 样本外锁死 + walk-forward + 防数据泄漏 三重保障，防止过拟合伪装成进化
- 影子运行当真正的裁判 开发集数字不算数，影子期表现才是真实裁判
- Deflated Sharpe 校正多重检验 防止大量搜索中的统计幸存者偏差

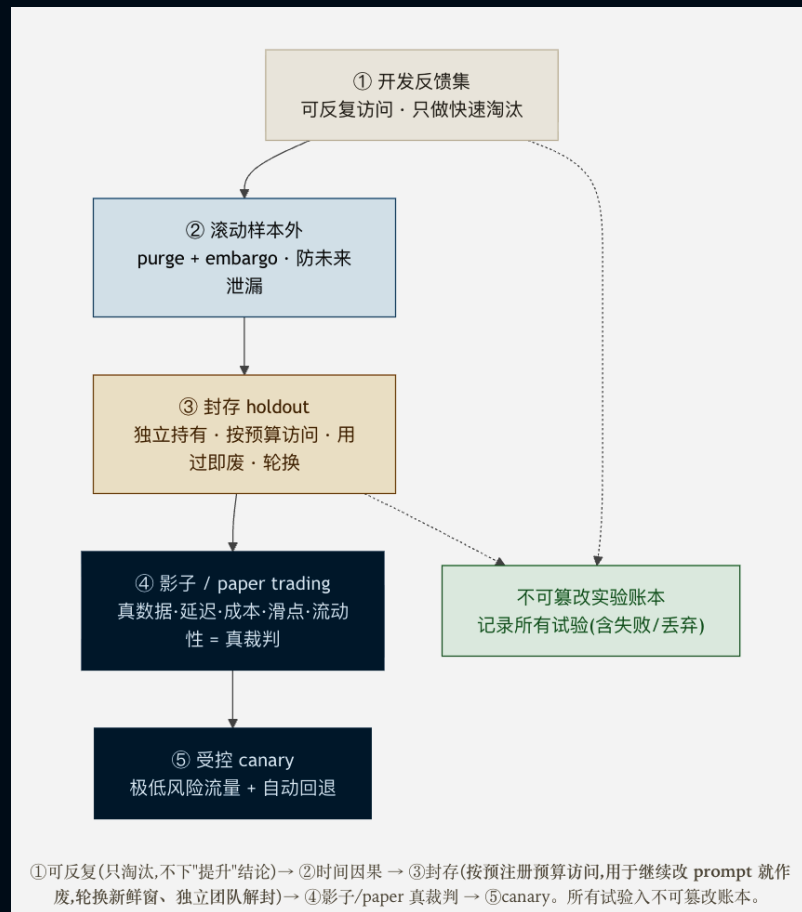
不足也要讲清楚

LLM 不是万能的候选生成器

arXiv:2603.24647：经典超参优化器在部分场景仍赢过 LLM。
LLM 更适合做候选生成器，而非最终裁判。

回测不是天然 ground truth

冻结验证集会慢慢退化成训练集。验证口径本身需要定期审视，但不能频繁变动。



三：金融原生的 Generative UI

通用 GenUI 的局限

通用 GenUI 给的是柱状图、饼图——这是数据可视化的最小公约数，不是金融分析师的工作语言。

金融要的是

价格与行情

K 线、收益率曲线

风险与绩效

回撤图、归因瀑布

因子分析

IC 热力图

全套金融控件

可拖拽 + 参数配置 分析师可以自由组合控件，调整参数，无需写代码

Agent 用空间 json 创建控件 描述布局 and 控件类型，不生成数值

Agent 用数据 json 更新 触发确定性管线取数，结果填入控件

Agent 的职责是编排控件，不是生成金融数值。数值由确定性管线负责。

✓ 生产实证

关键：Agent 不造数

数据研究应用三段式架构，中间段是要害：

参数进、数值出，不需要 Agent 介入。确定性管线计算所有金融数值：收益率、回撤、IC、归因等。幻觉进不了数据层。



数据源

行情数据库、因子库、交易记录。数据的真实性由数据源保证，不经过 Agent 加工。

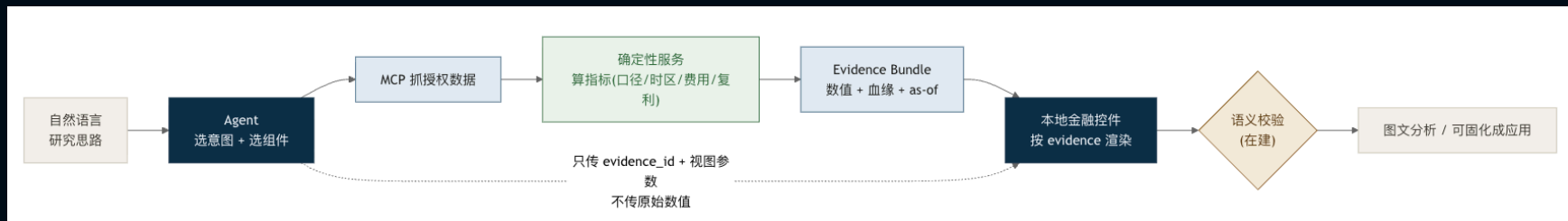
呈现

Agent 编排控件布局，解读结果含义。Agent 只做这两件事，不生成任何金融数值。

处理逻辑（要害）

参数进、数值出，不需要 Agent 介入。确定性管线计算所有金融数值：收益率、回撤、IC、归因等。幻觉进不了数据层。

从一句话到可复用的分析应用



场景：宏观研究全链路。从自然语言描述到可分享的分析应用，全程五步：



自然语言描述研究思路

分析师用自然语言描述研究意图，无需写代码或配置参数



Agent 通过 MCP 取数

Agent 编排数据获取流程，通过 MCP 协议调用数据源，触发确定性管线



金融控件呈现分析结果

K 线、收益率曲线、归因瀑布等金融原生控件自动组合，呈现分析结果



固化为个人分析应用

一键固化为可复用的分析应用，参数可调，逻辑可维护



按角色权限分享

使用者改参数；协作者/维护者改取数逻辑。权限分层，职责清晰

实话：幻觉缩小了，但没有消除

语义校验器在建

✅ 做到的

幻觉从"任意代码/数据"缩小为"可枚举、可验证、可回退的语义选择错误"。这是一个量级的改善，但不是消除。

❌ 没做到的

消除幻觉。这不是工程问题，是当前模型能力的边界。诚实面对这个边界，比假装消除更重要。

在建的下一步

- **语义校验器** 归因瀑布对不上账 → 直接拒绝渲染。不让错误的数字出现在界面上
- **共享应用统一血缘** 每一个分析应用的数据来源、处理逻辑全程可追溯

兜底机制

研究区 L0 只读、人在环，错误进不了交易。上线流程是那道闸，签名通道是最后防线。

04

工程心得分享

正在硬刚的五个难题

本场核心

这五个难题是我们正在解决的问题，它们比已解决的问题更有参考价值。

验证器设计比想象中难

业务指标无标准化数据集，大量自动搜索本质是**过拟合放大器**。这是我们没敢放开自进化规模的原因。验证器的设计质量决定了自进化能走多远。

人工采纳是隐形的税

Owner 把关防失控，但 Owner 时间成本会吃掉效率红利。**平衡点不好找**。自动化程度越高，人工审核的价值越难量化，但越不能省。

开源治理比想象中重

每个依赖都要 fork、审计、合规、跟上游。**纪律要求比纯自研还高**。开源的自由度是有代价的，代价是治理成本。

双区边界容易松

"为了赶上线"的每次松动都可能成为系统性风险源头。**必须靠签名通道的工程刚性守住**，不能靠人的自律。

GenUI 的领域真实性

涨跌颜色、年化算法、报告骨架不能让模型即兴发挥。**组件层守金融惯例与 Agent 创造力之间的张力要长期打磨**，没有一劳永逸的解法。

值得分享的五句话

- 划边界比选模型重要

边界划错不可逆，模型可以换。先把边界想清楚，再谈模型选型。这个顺序不能反。

- 玻璃箱 v2 管边界不管步骤

够强放自主，够弱留刚性。不是"结构越少越好"，是"能力到哪里，自主到哪里"。

- 可验证的指标是自进化的根基

验证器不能被作弊，比循环本身更要紧。没有可信的验证器，自进化就是过拟合的自动化。

- 开源治理走上游优先 + 短差异 fork + 动态 BOM + 准入闸

不走长期私有 fork。开源红利是"试得起"，不是"拿来就上"。治理纪律是使用开源的前提。

- 进化只在 prompt / skill / 路由这一层，不动模型权重

搜索空间的约束是金融场景的必要条件，不是保守主义。方向一致：模型变强后，架构更轻、更自主、边界更清晰。不只在金融适用。

谢谢

今天讲了什么

-  生产实证：双区架构、四级自治、签名通道、金融原生 GenUI
-  设计早期：编排做薄、受约束自进化、语义校验器、统一审计链
-  五个正在硬刚的难题

欢迎追问

边界判断的细节、验证器的设计取舍、开源治理的具体做法、GenUI 金融惯例的落地——任何一个话题都可以深聊。

也欢迎交流同行在类似场景下的不同选择。